Computer Architecture Laboratory Report

Teacher In-Charge

Asst. Prof. Chinmoy Ghorai

Name: Subhranil Ghoshal
Roll: 002210701122
Year: UG4
Semester: 1st
Session: 2025-26

Contents

1 Binary Multiplier Circuits **2**

2 Binary Divider Circuits **15**

Experiment - 01
Binary Multiplier Circuits

Objective

Design and implement binary multiplier circuits for both signed and unsigned numbers. Verify the functionality of these circuits using test cases for different inputs. Document your design approach and provide a detailed explanation of the multiplier's operation.

Tools Used

- **Language**: Verilog
- **Compiler**: Vivado 2025.1

Design

For this task, we design a parameterizable single-cycle multiplier which is purely combinational in nature.

Hardware implementations of multipliers which obey algorithms like Booth's algorithm or the classic shift-and-add algorithm require less hardware but more number of clock cycles, which increase linearly with the number of bits used to represent the operands.

Single cycle multipliers, on the other hand, use more hardware to perform faster multiplication. This improves throughput greatly, but also imposes additional restrictions on clock speed due to the circuit's large delay. This can be solved by using pipelining.

For the given task, we implement an array multiplier, which generates partial products by multiplying the multiplicand with each bit of the multiplier separately - an AND operation - and adds these partial products with appropriate shifts to produce the result of multiplication. The shifts, in turn, are implemented through appropriate wire connections only.

Schematic Diagrams

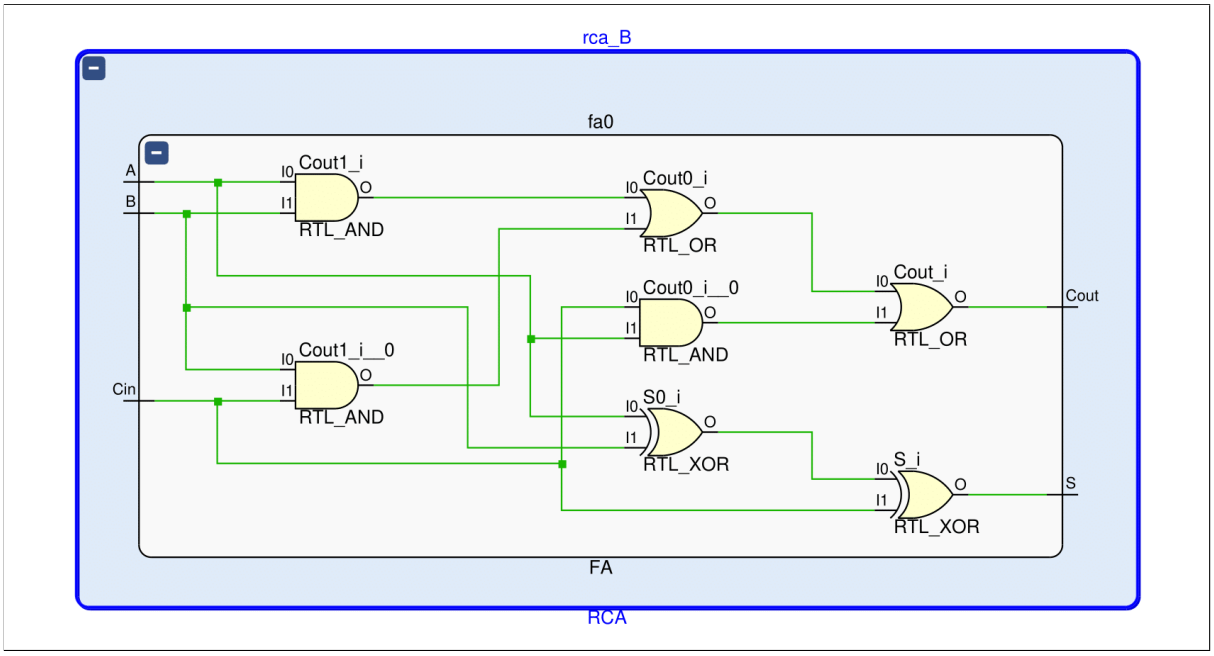

Figure 1.1: Full Adder

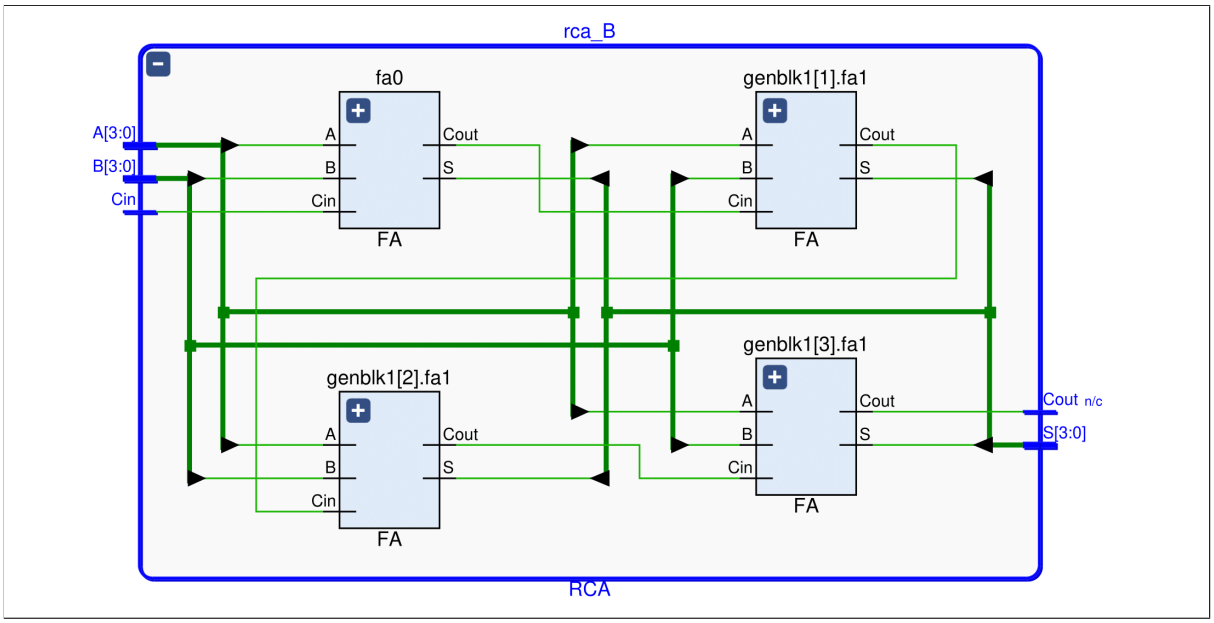

Figure 1.2: Parameterizable Ripple-Carry Adder

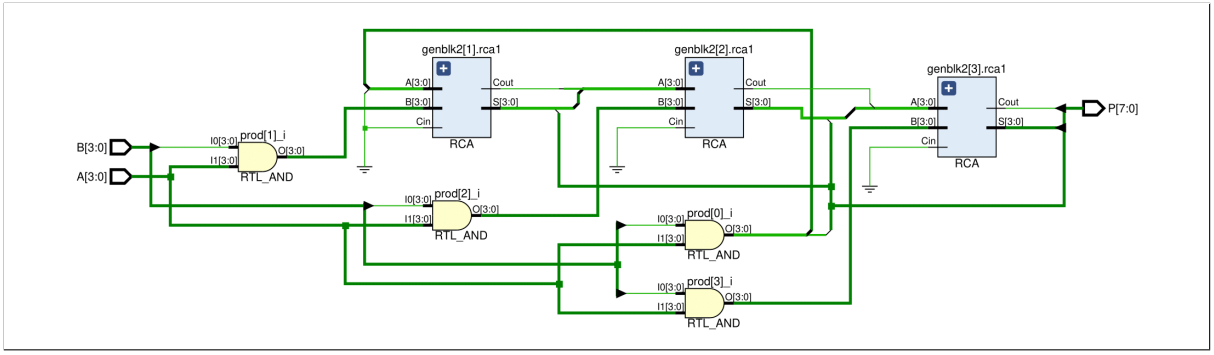

Figure 1.3: 4-bit Unsigned Multiplier

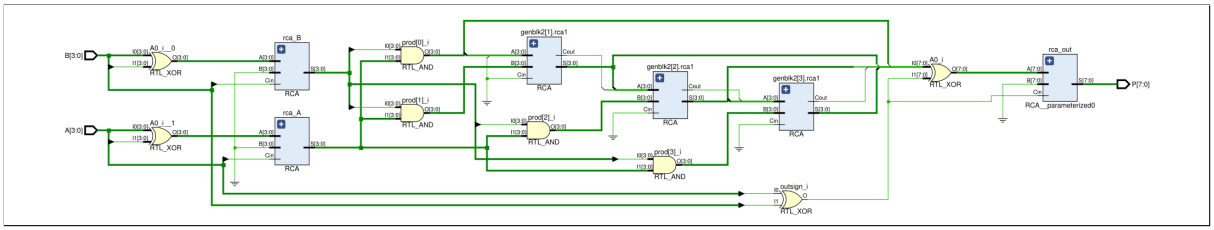

Figure 1.4: 4-bit Signed Multiplier

Verilog Code

Here, we include both design and testbench code.

i. Design

```
`timescale 1ns/1ps

module FA(
    input A, B, Cin,
    output S, Cout
);
    assign S = A^B^Cin;
    assign Cout = (A&B) | (B&Cin) | (Cin&A);
endmodule

module RCA  #(
    parameter N = 4
)(
    input [N-1:0] A, B,
    input Cin,
    output [N-1:0] S,
    output Cout
);
    genvar i;
    wire [N-1:0] C;
```

```
    assign Cout = C[N-1];
    FA fa0(.A(A[0]), .B(B[0]), .Cin(Cin), .S(S[0]), .Cout(C[0]));
    generate
        for(i=1; i<N; i=i+1) begin
            FA fa1(
                .A(A[i]),
                .B(B[i]),
                .Cin(C[i-1]),
                .S(S[i]),
                .Cout(C[i])
            );
        end
    endgenerate
endmodule

module Multiplier #(
    parameter N = 4
)(
    input [N-1:0] A, B,
    output [2*N-1:0] P
);
    genvar i, j;
    wire [N-1:0] prod[0:N-1];
    wire [N-1:0] partial[0:N-1];

    generate
        for(i=0; i<N; i=i+1) begin
            assign prod[i] = {N{B[i]}} & A;
        end
        assign partial[0] = {0, prod[0][N-1:1]};
        assign P[0] = prod[0][0];

        for(i=1; i<N; i=i+1) begin
            RCA #(.N(N)) rca1(
                .A(partial[i-1]),
                .B(prod[i]),
                .Cin(1'b0),
                .S({partial[i][N-2:0], P[i]}),
                .Cout(partial[i][N-1])
            );
        end
    endgenerate
    assign P[2*N-1:N] = partial[N-1];
endmodule

module signedMultiplier #(
    parameter N = 4
```

```
)(
    input [N-1:0] A, B,
    output [2*N-1:0] P
);

    genvar i, j;
    wire [N-1:0] Amag, Bmag;
    wire A_cout, B_cout, P_cout, outsign;
    wire [2*N-1:0] Pmag;

    // take 2's complement and add 1 if sign bit is set
    RCA #(.N(N)) rca_A(
        .A(A^{N{A[N-1]}}),
        .B('b0),
        .Cin(A[N-1]),
        .S(Amag),
        .Cout(A_cout)
    );
    RCA #(.N(N)) rca_B(
        .A(B^{N{B[N-1]}}),
        .B('b0),
        .Cin(B[N-1]),
        .S(Bmag),
        .Cout(B_cout)
    );
    assign outsign = A[N-1]^B[N-1];

    wire [N-1:0] prod[0:N-1];
    wire [N-1:0] partial[0:N-1];

    generate
        for(i=0; i<N; i=i+1) begin
            assign prod[i] = {N{Bmag[i]}} & Amag;
        end
        assign partial[0] = {0, prod[0][N-1:1]};
        assign Pmag[0] = prod[0][0];

        for(i=1; i<N; i=i+1) begin
            RCA #(.N(N)) rca1(
                .A(partial[i-1]),
                .B(prod[i]),
                .Cin(1'b0),
                .S({partial[i][N-2:0], Pmag[i]}),
                .Cout(partial[i][N-1])
            );
        end
    endgenerate
```

```
    assign Pmag[2*N-1:N] = partial[N-1];
    RCA #(.N(2*N)) rca_out(
        .A(Pmag^{2*N{outsign}}),
        .B('b0),
        .Cin(outsign),
        .S(P),
        .Cout(P_cout)
    );
endmodule
```

ii. Testbench

```
`timescale 1ns/1ps

module multiplier_tb();
    reg [3:0] A = 4'b0000;
    reg [3:0] B = 4'b0000;
    reg signed [3:0] sA = 4'b0000;
    reg signed [3:0] sB = 4'b0000;
    wire [7:0] P;
    wire signed [7:0] sP;
    Multiplier #(.N(4)) DUT(.A(A), .B(B), .P(P));
    signedMultiplier #(.N(4)) sDUT(.A(A), .B(B), .P(sP));

    initial begin
        $monitor("A: %d (%d), B: %d (%d), P: %d (%d)",
            A, sA, B, sB, P, sP);
        for(integer i=0; i<16; i=i+1) begin
            for(integer j=0; j<16; j=j+1) begin
                #1 A <= i; B <= j; sA <= i; sB <= j;
            end
        end
    #1 $finish;
    end
endmodule
```

Testbench Results

```
A:  0 (  0), B:  0 (  0), P:   0 (   0)
```

```
A:  0 (  0), B:  1 (  1), P:   0 (   0)
A:  0 (  0), B:  2 (  2), P:   0 (   0)
A:  0 (  0), B:  3 (  3), P:   0 (   0)
A:  0 (  0), B:  4 (  4), P:   0 (   0)
A:  0 (  0), B:  5 (  5), P:   0 (   0)
A:  0 (  0), B:  6 (  6), P:   0 (   0)
A:  0 (  0), B:  7 (  7), P:   0 (   0)
A:  0 (  0), B:  8 ( -8), P:   0 (   0)
A:  0 (  0), B:  9 ( -7), P:   0 (   0)
A:  0 (  0), B: 10 ( -6), P:   0 (   0)
A:  0 (  0), B: 11 ( -5), P:   0 (   0)
A:  0 (  0), B: 12 ( -4), P:   0 (   0)
A:  0 (  0), B: 13 ( -3), P:   0 (   0)
A:  0 (  0), B: 14 ( -2), P:   0 (   0)
A:  0 (  0), B: 15 ( -1), P:   0 (   0)
A:  1 (  1), B:  0 (  0), P:   0 (   0)
A:  1 (  1), B:  1 (  1), P:   1 (   1)
A:  1 (  1), B:  2 (  2), P:   2 (   2)
A:  1 (  1), B:  3 (  3), P:   3 (   3)
A:  1 (  1), B:  4 (  4), P:   4 (   4)
A:  1 (  1), B:  5 (  5), P:   5 (   5)
A:  1 (  1), B:  6 (  6), P:   6 (   6)
A:  1 (  1), B:  7 (  7), P:   7 (   7)
A:  1 (  1), B:  8 ( -8), P:   8 (  -8)
A:  1 (  1), B:  9 ( -7), P:   9 (  -7)
A:  1 (  1), B: 10 ( -6), P:  10 (  -6)
A:  1 (  1), B: 11 ( -5), P:  11 (  -5)
A:  1 (  1), B: 12 ( -4), P:  12 (  -4)
A:  1 (  1), B: 13 ( -3), P:  13 (  -3)
A:  1 (  1), B: 14 ( -2), P:  14 (  -2)
A:  1 (  1), B: 15 ( -1), P:  15 (  -1)
A:  2 (  2), B:  0 (  0), P:   0 (   0)
A:  2 (  2), B:  1 (  1), P:   2 (   2)
A:  2 (  2), B:  2 (  2), P:   4 (   4)
A:  2 (  2), B:  3 (  3), P:   6 (   6)
A:  2 (  2), B:  4 (  4), P:   8 (   8)
A:  2 (  2), B:  5 (  5), P:  10 (  10)
A:  2 (  2), B:  6 (  6), P:  12 (  12)
A:  2 (  2), B:  7 (  7), P:  14 (  14)
A:  2 (  2), B:  8 ( -8), P:  16 ( -16)
A:  2 (  2), B:  9 ( -7), P:  18 ( -14)
A:  2 (  2), B: 10 ( -6), P:  20 ( -12)
A:  2 (  2), B: 11 ( -5), P:  22 ( -10)
A:  2 (  2), B: 12 ( -4), P:  24 (  -8)
A:  2 (  2), B: 13 ( -3), P:  26 (  -6)
A:  2 (  2), B: 14 ( -2), P:  28 (  -4)
```

```
A:  2 (  2), B: 15 ( -1), P:  30 (  -2)
A:  3 (  3), B:  0 (  0), P:   0 (   0)
A:  3 (  3), B:  1 (  1), P:   3 (   3)
A:  3 (  3), B:  2 (  2), P:   6 (   6)
A:  3 (  3), B:  3 (  3), P:   9 (   9)
A:  3 (  3), B:  4 (  4), P:  12 (  12)
A:  3 (  3), B:  5 (  5), P:  15 (  15)
A:  3 (  3), B:  6 (  6), P:  18 (  18)
A:  3 (  3), B:  7 (  7), P:  21 (  21)
A:  3 (  3), B:  8 ( -8), P:  24 ( -24)
A:  3 (  3), B:  9 ( -7), P:  27 ( -21)
A:  3 (  3), B: 10 ( -6), P:  30 ( -18)
A:  3 (  3), B: 11 ( -5), P:  33 ( -15)
A:  3 (  3), B: 12 ( -4), P:  36 ( -12)
A:  3 (  3), B: 13 ( -3), P:  39 (  -9)
A:  3 (  3), B: 14 ( -2), P:  42 (  -6)
A:  3 (  3), B: 15 ( -1), P:  45 (  -3)
A:  4 (  4), B:  0 (  0), P:   0 (   0)
A:  4 (  4), B:  1 (  1), P:   4 (   4)
A:  4 (  4), B:  2 (  2), P:   8 (   8)
A:  4 (  4), B:  3 (  3), P:  12 (  12)
A:  4 (  4), B:  4 (  4), P:  16 (  16)
A:  4 (  4), B:  5 (  5), P:  20 (  20)
A:  4 (  4), B:  6 (  6), P:  24 (  24)
A:  4 (  4), B:  7 (  7), P:  28 (  28)
A:  4 (  4), B:  8 ( -8), P:  32 ( -32)
A:  4 (  4), B:  9 ( -7), P:  36 ( -28)
A:  4 (  4), B: 10 ( -6), P:  40 ( -24)
A:  4 (  4), B: 11 ( -5), P:  44 ( -20)
A:  4 (  4), B: 12 ( -4), P:  48 ( -16)
A:  4 (  4), B: 13 ( -3), P:  52 ( -12)
A:  4 (  4), B: 14 ( -2), P:  56 (  -8)
A:  4 (  4), B: 15 ( -1), P:  60 (  -4)
A:  5 (  5), B:  0 (  0), P:   0 (   0)
A:  5 (  5), B:  1 (  1), P:   5 (   5)
A:  5 (  5), B:  2 (  2), P:  10 (  10)
A:  5 (  5), B:  3 (  3), P:  15 (  15)
A:  5 (  5), B:  4 (  4), P:  20 (  20)
A:  5 (  5), B:  5 (  5), P:  25 (  25)
A:  5 (  5), B:  6 (  6), P:  30 (  30)
A:  5 (  5), B:  7 (  7), P:  35 (  35)
A:  5 (  5), B:  8 ( -8), P:  40 ( -40)
A:  5 (  5), B:  9 ( -7), P:  45 ( -35)
A:  5 (  5), B: 10 ( -6), P:  50 ( -30)
A:  5 (  5), B: 11 ( -5), P:  55 ( -25)
A:  5 (  5), B: 12 ( -4), P:  60 ( -20)
```

```
A:  5 (  5), B: 13 ( -3), P:  65 ( -15)
A:  5 (  5), B: 14 ( -2), P:  70 ( -10)
A:  5 (  5), B: 15 ( -1), P:  75 (  -5)
A:  6 (  6), B:  0 (  0), P:   0 (   0)
A:  6 (  6), B:  1 (  1), P:   6 (   6)
A:  6 (  6), B:  2 (  2), P:  12 (  12)
A:  6 (  6), B:  3 (  3), P:  18 (  18)
A:  6 (  6), B:  4 (  4), P:  24 (  24)
A:  6 (  6), B:  5 (  5), P:  30 (  30)
A:  6 (  6), B:  6 (  6), P:  36 (  36)
A:  6 (  6), B:  7 (  7), P:  42 (  42)
A:  6 (  6), B:  8 ( -8), P:  48 ( -48)
A:  6 (  6), B:  9 ( -7), P:  54 ( -42)
A:  6 (  6), B: 10 ( -6), P:  60 ( -36)
A:  6 (  6), B: 11 ( -5), P:  66 ( -30)
A:  6 (  6), B: 12 ( -4), P:  72 ( -24)
A:  6 (  6), B: 13 ( -3), P:  78 ( -18)
A:  6 (  6), B: 14 ( -2), P:  84 ( -12)
A:  6 (  6), B: 15 ( -1), P:  90 (  -6)
A:  7 (  7), B:  0 (  0), P:   0 (   0)
A:  7 (  7), B:  1 (  1), P:   7 (   7)
A:  7 (  7), B:  2 (  2), P:  14 (  14)
A:  7 (  7), B:  3 (  3), P:  21 (  21)
A:  7 (  7), B:  4 (  4), P:  28 (  28)
A:  7 (  7), B:  5 (  5), P:  35 (  35)
A:  7 (  7), B:  6 (  6), P:  42 (  42)
A:  7 (  7), B:  7 (  7), P:  49 (  49)
A:  7 (  7), B:  8 ( -8), P:  56 ( -56)
A:  7 (  7), B:  9 ( -7), P:  63 ( -49)
A:  7 (  7), B: 10 ( -6), P:  70 ( -42)
A:  7 (  7), B: 11 ( -5), P:  77 ( -35)
A:  7 (  7), B: 12 ( -4), P:  84 ( -28)
A:  7 (  7), B: 13 ( -3), P:  91 ( -21)
A:  7 (  7), B: 14 ( -2), P:  98 ( -14)
A:  7 (  7), B: 15 ( -1), P: 105 (  -7)
A:  8 ( -8), B:  0 (  0), P:   0 (   0)
A:  8 ( -8), B:  1 (  1), P:   8 (  -8)
A:  8 ( -8), B:  2 (  2), P:  16 ( -16)
A:  8 ( -8), B:  3 (  3), P:  24 ( -24)
A:  8 ( -8), B:  4 (  4), P:  32 ( -32)
A:  8 ( -8), B:  5 (  5), P:  40 ( -40)
A:  8 ( -8), B:  6 (  6), P:  48 ( -48)
A:  8 ( -8), B:  7 (  7), P:  56 ( -56)
A:  8 ( -8), B:  8 ( -8), P:  64 (  64)
A:  8 ( -8), B:  9 ( -7), P:  72 (  56)
A:  8 ( -8), B: 10 ( -6), P:  80 (  48)
```

```
A:  8 ( -8), B: 11 ( -5), P:  88 (  40)
A:  8 ( -8), B: 12 ( -4), P:  96 (  32)
A:  8 ( -8), B: 13 ( -3), P: 104 (  24)
A:  8 ( -8), B: 14 ( -2), P: 112 (  16)
A:  8 ( -8), B: 15 ( -1), P: 120 (   8)
A:  9 ( -7), B:  0 (  0), P:   0 (   0)
A:  9 ( -7), B:  1 (  1), P:   9 (  -7)
A:  9 ( -7), B:  2 (  2), P:  18 ( -14)
A:  9 ( -7), B:  3 (  3), P:  27 ( -21)
A:  9 ( -7), B:  4 (  4), P:  36 ( -28)
A:  9 ( -7), B:  5 (  5), P:  45 ( -35)
A:  9 ( -7), B:  6 (  6), P:  54 ( -42)
A:  9 ( -7), B:  7 (  7), P:  63 ( -49)
A:  9 ( -7), B:  8 ( -8), P:  72 (  56)
A:  9 ( -7), B:  9 ( -7), P:  81 (  49)
A:  9 ( -7), B: 10 ( -6), P:  90 (  42)
A:  9 ( -7), B: 11 ( -5), P:  99 (  35)
A:  9 ( -7), B: 12 ( -4), P: 108 (  28)
A:  9 ( -7), B: 13 ( -3), P: 117 (  21)
A:  9 ( -7), B: 14 ( -2), P: 126 (  14)
A:  9 ( -7), B: 15 ( -1), P: 135 (   7)
A: 10 ( -6), B:  0 (  0), P:   0 (   0)
A: 10 ( -6), B:  1 (  1), P:  10 (  -6)
A: 10 ( -6), B:  2 (  2), P:  20 ( -12)
A: 10 ( -6), B:  3 (  3), P:  30 ( -18)
A: 10 ( -6), B:  4 (  4), P:  40 ( -24)
A: 10 ( -6), B:  5 (  5), P:  50 ( -30)
A: 10 ( -6), B:  6 (  6), P:  60 ( -36)
A: 10 ( -6), B:  7 (  7), P:  70 ( -42)
A: 10 ( -6), B:  8 ( -8), P:  80 (  48)
A: 10 ( -6), B:  9 ( -7), P:  90 (  42)
A: 10 ( -6), B: 10 ( -6), P: 100 (  36)
A: 10 ( -6), B: 11 ( -5), P: 110 (  30)
A: 10 ( -6), B: 12 ( -4), P: 120 (  24)
A: 10 ( -6), B: 13 ( -3), P: 130 (  18)
A: 10 ( -6), B: 14 ( -2), P: 140 (  12)
A: 10 ( -6), B: 15 ( -1), P: 150 (   6)
A: 11 ( -5), B:  0 (  0), P:   0 (   0)
A: 11 ( -5), B:  1 (  1), P:  11 (  -5)
A: 11 ( -5), B:  2 (  2), P:  22 ( -10)
A: 11 ( -5), B:  3 (  3), P:  33 ( -15)
A: 11 ( -5), B:  4 (  4), P:  44 ( -20)
A: 11 ( -5), B:  5 (  5), P:  55 ( -25)
A: 11 ( -5), B:  6 (  6), P:  66 ( -30)
A: 11 ( -5), B:  7 (  7), P:  77 ( -35)
A: 11 ( -5), B:  8 ( -8), P:  88 (  40)
```

```
A: 11 ( -5), B:  9 ( -7), P:  99 (  35)
A: 11 ( -5), B: 10 ( -6), P: 110 (  30)
A: 11 ( -5), B: 11 ( -5), P: 121 (  25)
A: 11 ( -5), B: 12 ( -4), P: 132 (  20)
A: 11 ( -5), B: 13 ( -3), P: 143 (  15)
A: 11 ( -5), B: 14 ( -2), P: 154 (  10)
A: 11 ( -5), B: 15 ( -1), P: 165 (   5)
A: 12 ( -4), B:  0 (  0), P:   0 (   0)
A: 12 ( -4), B:  1 (  1), P:  12 (  -4)
A: 12 ( -4), B:  2 (  2), P:  24 (  -8)
A: 12 ( -4), B:  3 (  3), P:  36 ( -12)
A: 12 ( -4), B:  4 (  4), P:  48 ( -16)
A: 12 ( -4), B:  5 (  5), P:  60 ( -20)
A: 12 ( -4), B:  6 (  6), P:  72 ( -24)
A: 12 ( -4), B:  7 (  7), P:  84 ( -28)
A: 12 ( -4), B:  8 ( -8), P:  96 (  32)
A: 12 ( -4), B:  9 ( -7), P: 108 (  28)
A: 12 ( -4), B: 10 ( -6), P: 120 (  24)
A: 12 ( -4), B: 11 ( -5), P: 132 (  20)
A: 12 ( -4), B: 12 ( -4), P: 144 (  16)
A: 12 ( -4), B: 13 ( -3), P: 156 (  12)
A: 12 ( -4), B: 14 ( -2), P: 168 (   8)
A: 12 ( -4), B: 15 ( -1), P: 180 (   4)
A: 13 ( -3), B:  0 (  0), P:   0 (   0)
A: 13 ( -3), B:  1 (  1), P:  13 (  -3)
A: 13 ( -3), B:  2 (  2), P:  26 (  -6)
A: 13 ( -3), B:  3 (  3), P:  39 (  -9)
A: 13 ( -3), B:  4 (  4), P:  52 ( -12)
A: 13 ( -3), B:  5 (  5), P:  65 ( -15)
A: 13 ( -3), B:  6 (  6), P:  78 ( -18)
A: 13 ( -3), B:  7 (  7), P:  91 ( -21)
A: 13 ( -3), B:  8 ( -8), P: 104 (  24)
A: 13 ( -3), B:  9 ( -7), P: 117 (  21)
A: 13 ( -3), B: 10 ( -6), P: 130 (  18)
A: 13 ( -3), B: 11 ( -5), P: 143 (  15)
A: 13 ( -3), B: 12 ( -4), P: 156 (  12)
A: 13 ( -3), B: 13 ( -3), P: 169 (   9)
A: 13 ( -3), B: 14 ( -2), P: 182 (   6)
A: 13 ( -3), B: 15 ( -1), P: 195 (   3)
A: 14 ( -2), B:  0 (  0), P:   0 (   0)
A: 14 ( -2), B:  1 (  1), P:  14 (  -2)
A: 14 ( -2), B:  2 (  2), P:  28 (  -4)
A: 14 ( -2), B:  3 (  3), P:  42 (  -6)
A: 14 ( -2), B:  4 (  4), P:  56 (  -8)
A: 14 ( -2), B:  5 (  5), P:  70 ( -10)
A: 14 ( -2), B:  6 (  6), P:  84 ( -12)
```

```
A: 14 ( -2), B:  7 (  7), P:  98 ( -14)
A: 14 ( -2), B:  8 ( -8), P: 112 (  16)
A: 14 ( -2), B:  9 ( -7), P: 126 (  14)
A: 14 ( -2), B: 10 ( -6), P: 140 (  12)
A: 14 ( -2), B: 11 ( -5), P: 154 (  10)
A: 14 ( -2), B: 12 ( -4), P: 168 (   8)
A: 14 ( -2), B: 13 ( -3), P: 182 (   6)
A: 14 ( -2), B: 14 ( -2), P: 196 (   4)
A: 14 ( -2), B: 15 ( -1), P: 210 (   2)
A: 15 ( -1), B:  0 (  0), P:   0 (   0)
A: 15 ( -1), B:  1 (  1), P:  15 (  -1)
A: 15 ( -1), B:  2 (  2), P:  30 (  -2)
A: 15 ( -1), B:  3 (  3), P:  45 (  -3)
A: 15 ( -1), B:  4 (  4), P:  60 (  -4)
A: 15 ( -1), B:  5 (  5), P:  75 (  -5)
A: 15 ( -1), B:  6 (  6), P:  90 (  -6)
A: 15 ( -1), B:  7 (  7), P: 105 (  -7)
A: 15 ( -1), B:  8 ( -8), P: 120 (   8)
A: 15 ( -1), B:  9 ( -7), P: 135 (   7)
A: 15 ( -1), B: 10 ( -6), P: 150 (   6)
A: 15 ( -1), B: 11 ( -5), P: 165 (   5)
A: 15 ( -1), B: 12 ( -4), P: 180 (   4)
A: 15 ( -1), B: 13 ( -3), P: 195 (   3)
A: 15 ( -1), B: 14 ( -2), P: 210 (   2)
A: 15 ( -1), B: 15 ( -1), P: 225 (   1)
```

Clearly, the results for both signed and unsigned multipliers produce accurate results for all possible input combinations.

Discussion

1. The multiplier circuit thus constructed is the single-cycle array multiplier, which is a purely combinational circuit. This, in theory, makes it faster than a multi-cycle multiplier.

2. The single-cycle multiplier may however not be faster in reality, due to restrictions imposed on clock frequency by the combination of its gate delays. Operations at higher frequencies will inevitably lead to timing violations, leading to incorrect outputs or even metastability within the system.

3. The architecture implemented can be made to operate at higher frequencies by introducing appropriate pipeline registers, at the cost of more area and hardware.

   Pipelining introduces a latency of several clock cycles, depending on its depth, but allows for the same throughput as non-pipelined architecture.

4. Faster single-cycle multiplier architectures are possible, e.g., Dadda multiplier,

Wallace tree multiplier, etc. However, these are not as easily parameterized, and require more hardware.

5. For small binary operands, ROM can be used instead of combinational logic to precompute and store the results of multiplications.

Conclusion

In this experiment, a parameterizable array multiplier has been implemented using Verilog HDL, with demonstration for the 4-bit multiplier. Such a multiplier can operate faster than a simple shift-and-add multiplier due to the latter's dependence on the length of binary operands. However, such multipliers inevitably require more area and consume more power due to a larger number of gates required.

Experiment - 02
Binary Divider Circuits

Objective

Tools Used

Design

Schematic Diagrams

Verilog Code

Testbench Results

Discussion

Conclusion